

" OrbitMind AI– Intelligent Productivity & Study Companion"

by

Md. Imroz Newaz

CSE 11220321052

Zeniya Naznin

CSE 11230121138

Fatema Parvin Kanta

CSE 11230121192

Md. Mahfuzur Rahman Hridoy

CSE 11230121186

WEEK 07 ASSIGNMENT

Course Title: Software Development III: AI-Based Software Project Development

Course Code: CSE 4104

Section: 7D | Spring 2026



Submitted to

Md. Riaz Mahmud

Assistant Professor

Department of Computer Science and Engineering

Northern University of Business and Technology, Khulna

Khulna 9100, Bangladesh

CONTENTS

Title Page	1
Contents	2
Figures	3
CHAPTER I – Team Information	4
1.1 Team Information Sheet	4
1.1.1 Team Members	4
CHAPTER II – Frontend Architecture & Integration Report	5-7
2.1 Frontend Technology Stack & Project Setup	5
2.2 Completed Pages & UI Implementation	5
2.3 Backend API Integration	5
2.4 Authentication Flow	6
2.5 Form Validation & Data Integrity	6
2.6 Responsive Design & User Experience	6
2.7 Current Development Progress	7
2.8 GitHub Repository Link	7
CHAPTER III – Visual Evidence & Implementation Screenshots	8-15
3.1 The Landing/Home Page	8
3.2 User Authentication (Registration & Login Flow)	8-9
3.3 Central Dashboard (Displaying Dynamic Backend Data)	10
3.4 Major Functional AI Pages	10-12
3.5 Responsive Mobile View	12-13
3.6 API Integration	13-14
3.7 Maintained GitHub Repository	14-15

FIGURES

1. Figure: The fully responsive OrbitMind AI Landing Page.	8
2. Figure: The secure Registration Interface with form validation	9
3. Figure: The Secure Login Interface handling JWT authentication	9
4. Figure: The Central Dashboard dynamically rendering data from the backend.	10
5. Figure: The Task Management interface for CRUD operations.	11
6. Figure: The AI Study Planner interface integrating with Gemini AI.	11
7. Figure: The AI Coach Chatbot interface.	12
8. Figure: The AI Note Summarizer interface for condensing academic material.	12
9. Figure: Mobile viewport simulation demonstrating responsive design scalability	13
10. Figure: Network validation confirming successful frontend-to-backend API	14
11. Figure: Maintained GitHub repository documenting frontend dev commits.	15

CHAPTER I

1.1 Team Information Sheet

Field	Details
Team Name	CSE4104-7D-T04
Section	7D
Project Title	OrbitMind AI — Smart Productivity & Study Assistant
Team Leader	Md. Imroz Newaz (ID: 11220321052)

1.1.1 Team Members

Member Name	Student ID	Role
Md. Imroz Newaz	11220321052	Team Leader & Database Manager
Zeniya Naznin	11230121138	Frontend Developer
Fatema Parvin Kanta	11230121192	Backend Developer
Md. Mahfuzur Rahman Hridoy	11230121186	AI Integration Lead

CHAPTER II

2.1 Frontend Technology Stack & Project Setup

For the frontend architecture, our team utilized **Vanilla JavaScript (ES6+), HTML5, and CSS3**. By bypassing heavy frameworks, we maintained an exceptionally clean, lightweight, and scalable project structure without the overhead of virtual DOM dependencies. Our frontend directory is strictly organized into */css*, */js*, and HTML views, with logic decoupled into modular files (e.g., *api.js* for global fetch wrappers).

2.2 Completed Pages & UI Implementation

We successfully transformed the high-fidelity UI/UX designs from Week 05 into fully functional web interfaces. The completed screens include:

- **Landing/Home Page:** A responsive, visually engaging introduction.
- **Authentication Hub:** Secure Login and Registration pages.
- **Central Dashboard:** A comprehensive overview of user progress.
- **Core Functional Pages:** Task Manager and User Profile interfaces.
- **AI Interaction Pages:** The AI Study Planner, AI Coach Chatbot, and Note Summarizer modules.

2.3 Backend API Integration

We have fully connected the frontend UI with our Week 06 backend Express.js APIs. Our custom *apiFetch()* wrapper ensures that data is loaded dynamically from the MongoDB Atlas database rather than relying on static or hardcoded HTML values.

- **CRUD Operations:** Users can create, read, update, and delete tasks directly from the UI.
- **AI Request Submission:** Complex user prompts are gathered from the frontend, routed securely to our backend controllers, and processed via the Google Gemini API integration.

2.4 Authentication Flow

A highly secure, stateless authentication flow has been implemented.

- **Session Management:** Upon successful login, the backend issues a JSON Web Token (JWT). The frontend stores this token in localStorage.
- **Protected Routes:** We developed a checkAuth() middleware function on the frontend. Unauthenticated attempts to access the dashboard are intercepted, wiping invalid session data and redirecting to the login screen.
- **User Logout:** A secure logout function clears the session data and terminates access.

2.5 Form Validation & Data Integrity

To prevent invalid data submission, we implemented rigorous client-side form validation:

- **Required Fields:** All forms strictly enforce mandatory inputs.
- **Format Validation:** Registration enforces email format validation and minimum password lengths.
- **User-Friendly Error Handling:** If validation fails or the backend returns an error (e.g., "Invalid Credentials"), the UI intercepts the error and displays a clear, actionable alert.

2.6 Responsive Design & User Experience

The application was built with a mobile-first responsive design philosophy, functioning flawlessly across Desktop, Tablet, and Mobile viewports. To enhance the overall usability, we implemented several micro-interactions:

- **Loading Indicators:** Buttons display visual loading states during API calls.
- **Feedback Messages:** Success alerts and error messages are rendered dynamically.
- **Empty-State Messages:** The UI gracefully displays "Empty-state" graphics when users have no active tasks.
- **Theming:** A persistent Dark/Light mode toggle reduces eye strain.

2.7 Current Development Progress

We are proud to report that the Frontend Development and Backend Integration phase (Week 07) is 100% complete. The frontend successfully communicates with the Node.js backend APIs, all dynamic data is rendering correctly on the UI, and the stateless JWT authentication flow is completely secure. The application is fully responsive and boasts a highly polished user experience. Our next phase will focus on final system testing and deployment preparation.

2.8 GitHub Repository Link

Our team continues to maintain an organized repository with meaningful, descriptive commit messages documenting our frontend development cycle.

Repository	Link
Official GitHub Repo	https://github.com/Mahfuz-Hridoy/cse4104-7d-t04-orbitmind-ai.git

CHAPTER III

3.1 The Landing/Home Page

Implementation Detail: This screenshot demonstrates the fully responsive landing page. It utilizes semantic HTML5 and CSS3 to deliver an engaging, lightweight entry point to the OrbitMind AI platform, adhering strictly to the Week 05 UI/UX designs.

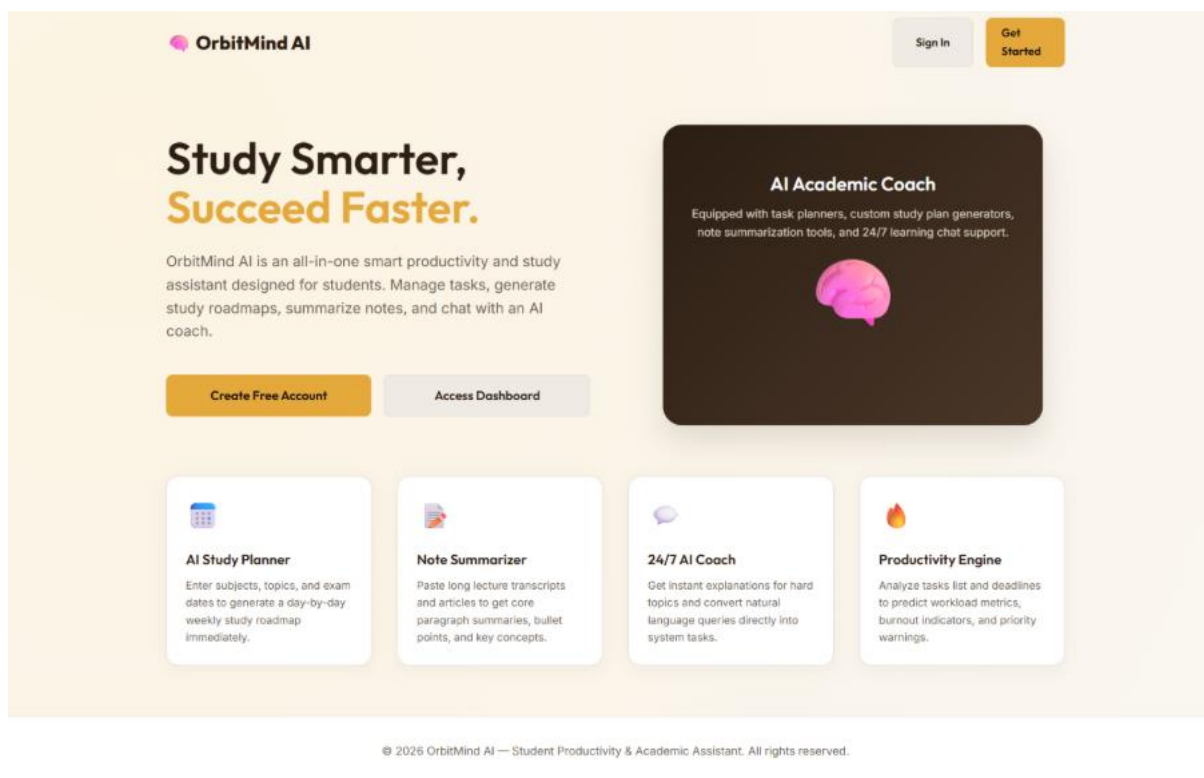
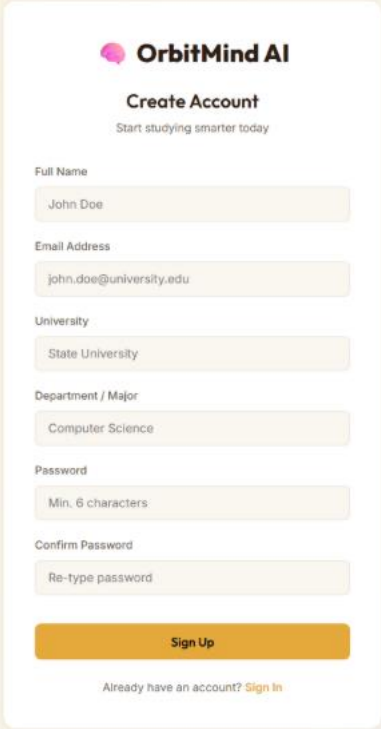


Figure 1: The fully responsive OrbitMind AI Landing Page.

3.2 User Authentication (Registration & Login Flow)

Implementation Detail: These interfaces showcase our secure authentication hub. The forms feature real-time client-side validation to ensure data integrity before submission. Upon successful login, the frontend dynamically intercepts the JWT token provided by the Node.js backend and securely stores it in the browser's local storage for session management.



The image shows a 'Create Account' form for OrbitMind AI. The form is centered on a light orange background. It includes fields for Full Name (John Doe), Email Address (john.doe@university.edu), University (State University), Department / Major (Computer Science), Password (Min. 6 characters), and Confirm Password (Re-type password). A yellow 'Sign Up' button is at the bottom, with a link to 'Sign In' for existing users.

OrbitMind AI

Create Account
Start studying smarter today

Full Name
John Doe

Email Address
john.doe@university.edu

University
State University

Department / Major
Computer Science

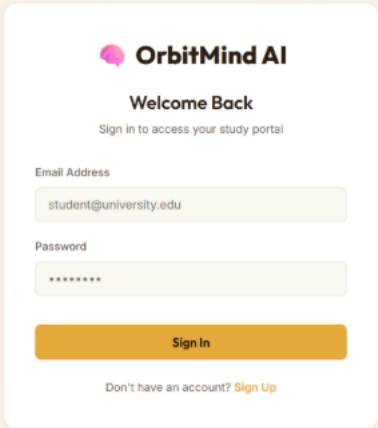
Password
Min. 6 characters

Confirm Password
Re-type password

Sign Up

Already have an account? [Sign In](#)

Figure 2: The secure Registration Interface with form validation.



The image shows a 'Welcome Back' login form for OrbitMind AI. The form is centered on a light orange background. It includes fields for Email Address (student@university.edu) and Password (masked with dots). A yellow 'Sign In' button is at the bottom, with a link to 'Sign Up' for new users.

OrbitMind AI

Welcome Back
Sign in to access your study portal

Email Address
student@university.edu

Password

Sign In

Don't have an account? [Sign Up](#)

Figure 3: Secure Login Interface handling JWT authentication.

3.3 Central Dashboard (Displaying Dynamic Backend Data)

Implementation Detail: This screenshot proves successful backend integration. The dashboard is entirely devoid of hardcoded data; instead, it dynamically fetches and renders the authenticated user's profile information, upcoming task deadlines, and recent notifications directly from the MongoDB Atlas database via our custom *apiFetch()* wrapper.

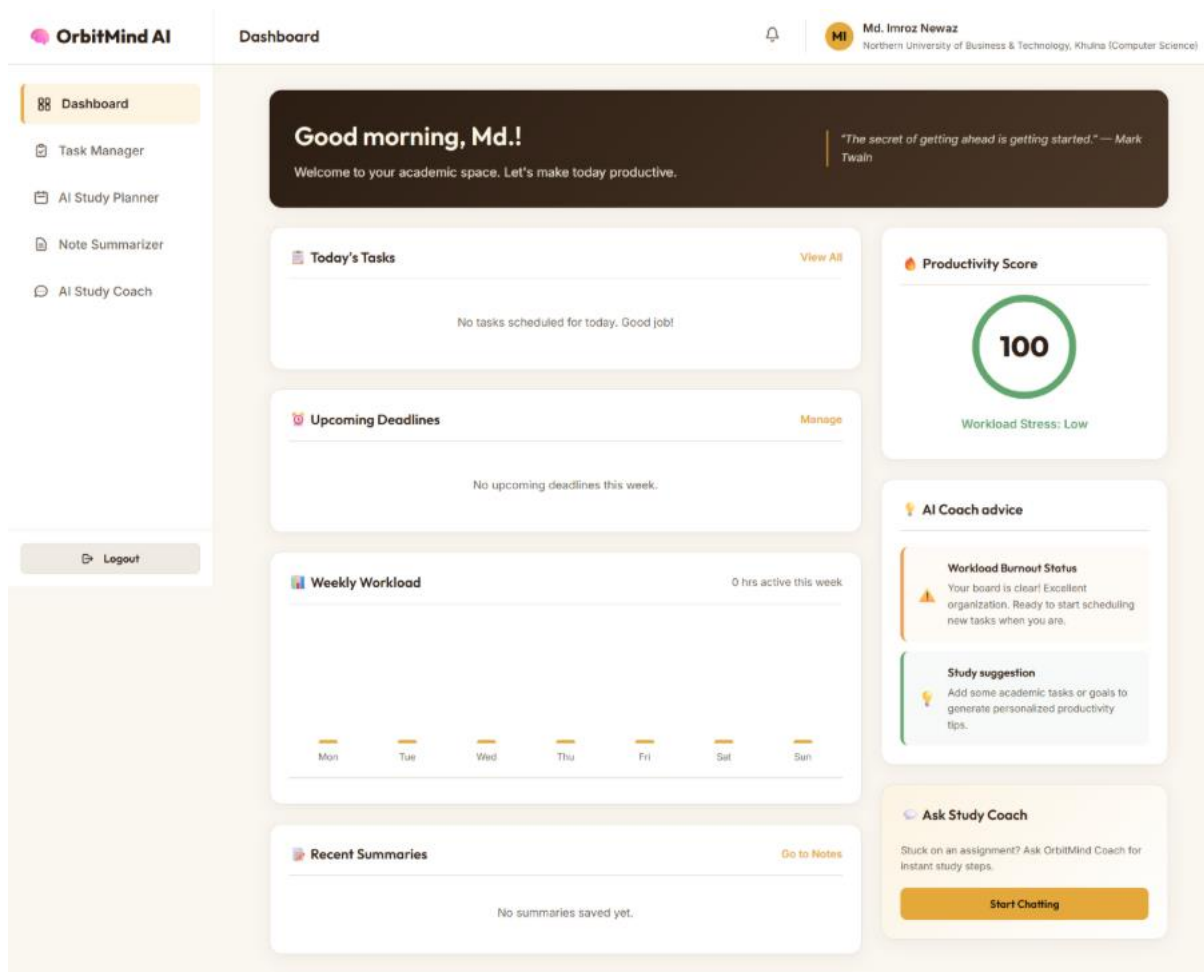


Figure 4: The Central Dashboard dynamically rendering data from the backend.

3.4 Major Functional AI Pages

Implementation Detail: Displayed here are the core functional modules of the application. These interfaces allow users to perform CRUD operations on their tasks and securely submit various academic prompts to the Google Gemini AI. The UI features dynamic loading indicators and empty-state graphics to significantly enhance the overall user experience.

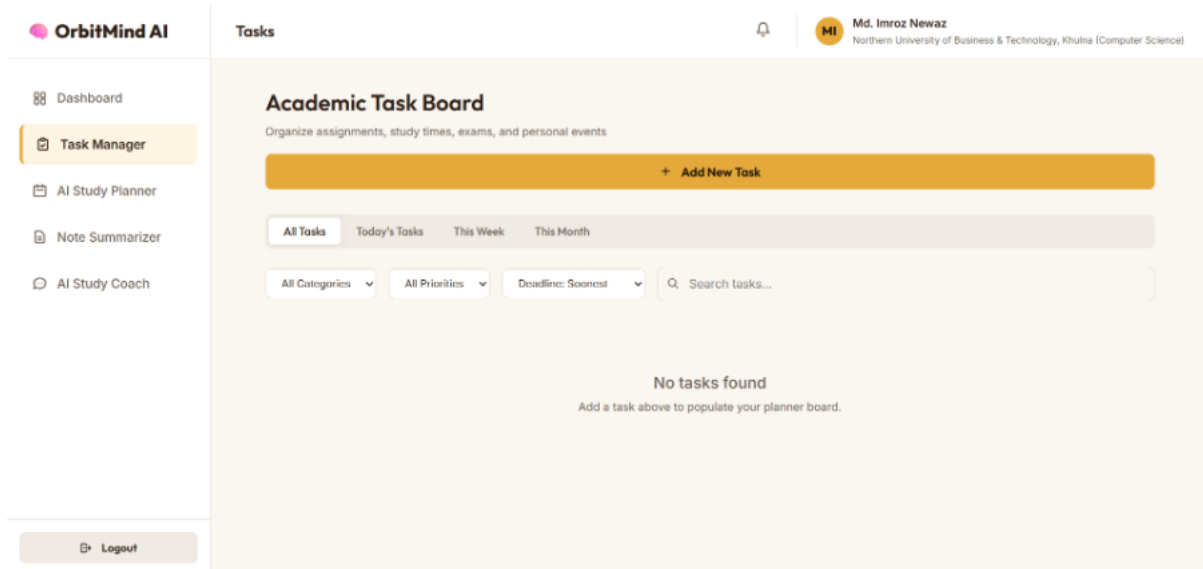


Figure 5: The Task Management interface for CRUD operations.

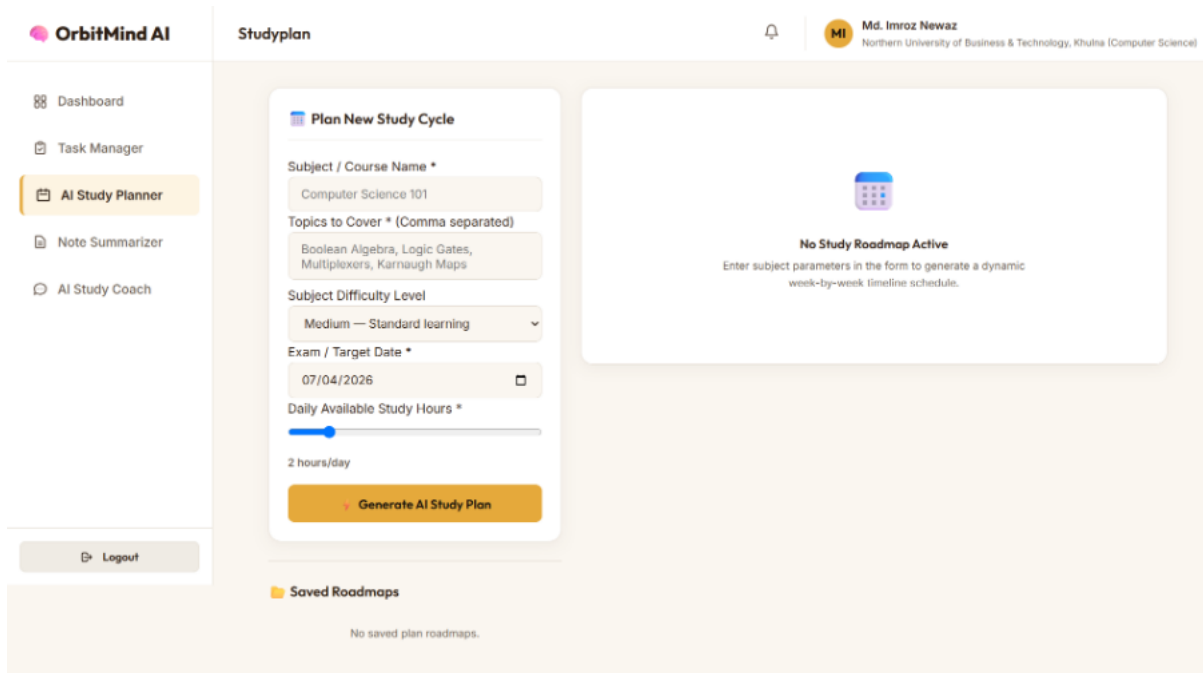


Figure 6: The AI Study Planner interface integrating with Gemini AI.

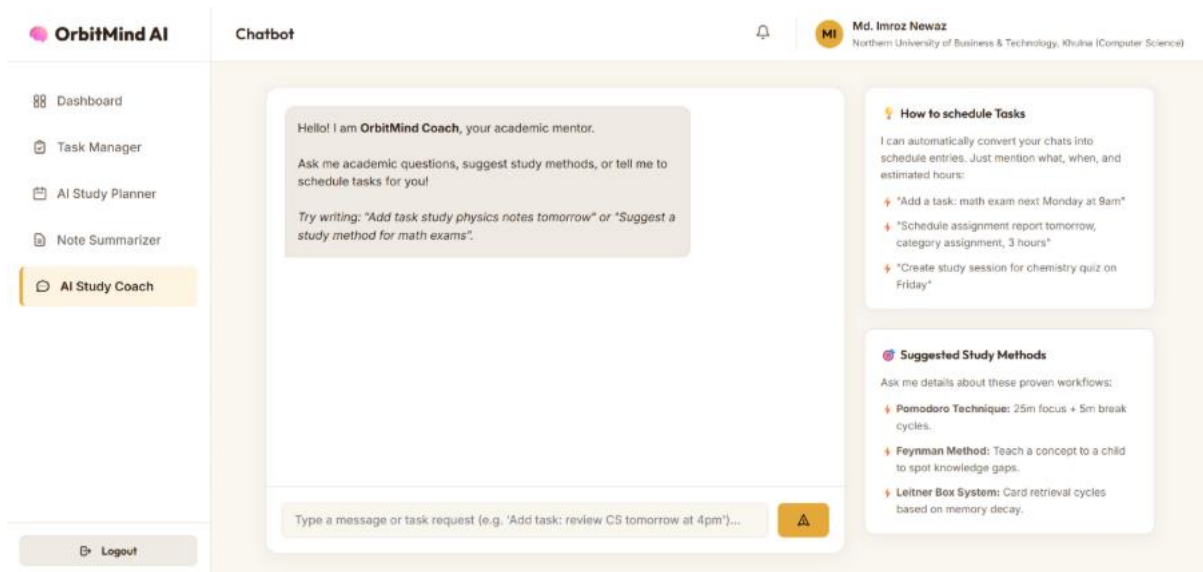


Figure 7: The AI Coach Chatbot interface.

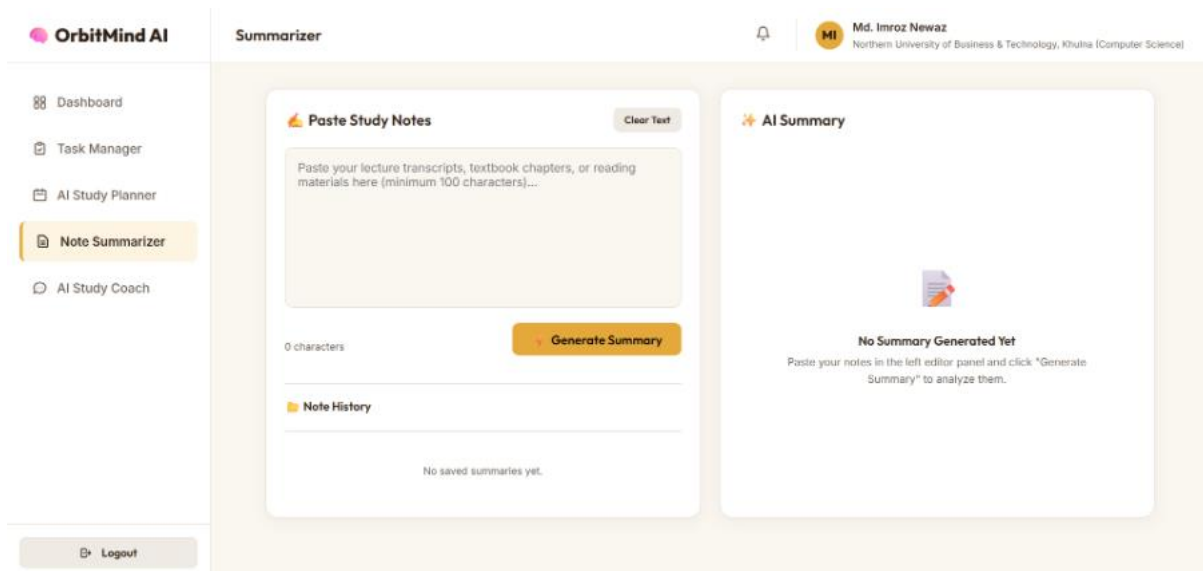


Figure 8: The AI Note Summarizer interface for condensing academic material.

3.5 Responsive Mobile View

Implementation Detail: This screenshot highlights our mobile-first CSS architecture applied to a complex functional page. The Task Management layout, form inputs, and interactive task cards fluidly stack and adapt to smaller screen sizes. This ensures the application remains perfectly usable, readable, and accessible on mobile devices without horizontal scrolling or broken elements.

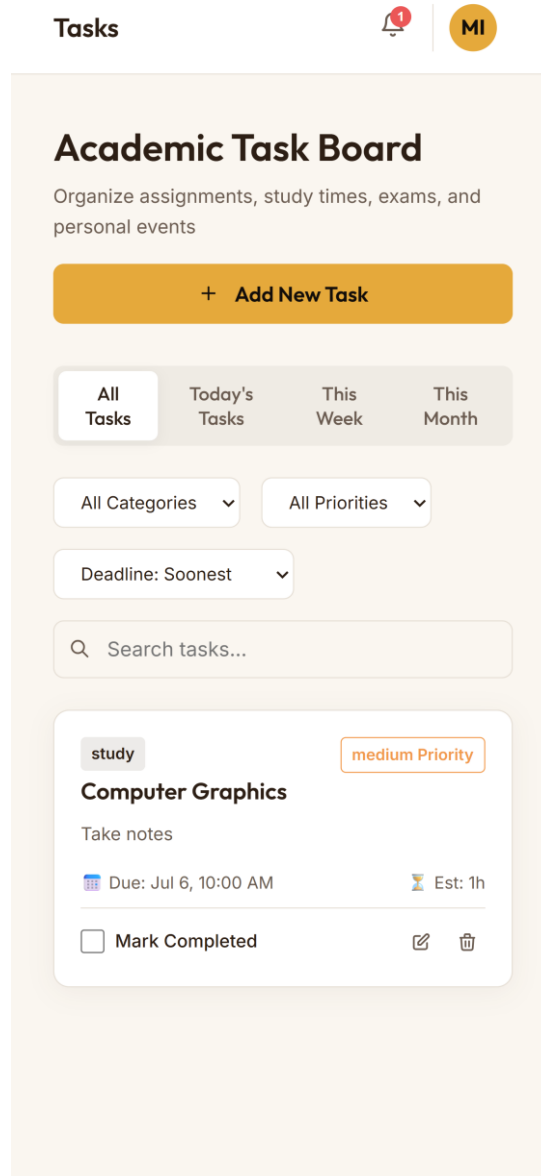


Figure 9: Mobile viewport simulation demonstrating responsive design scalability on the Task Management interface.

3.6 API Integration

Implementation Detail: This visual confirms the successful handshake between our Vanilla JavaScript frontend and the Express.js backend. It demonstrates a secure, authorized HTTP request returning a 200 OK status, confirming that our protected routes and JWT middleware are functioning correctly.

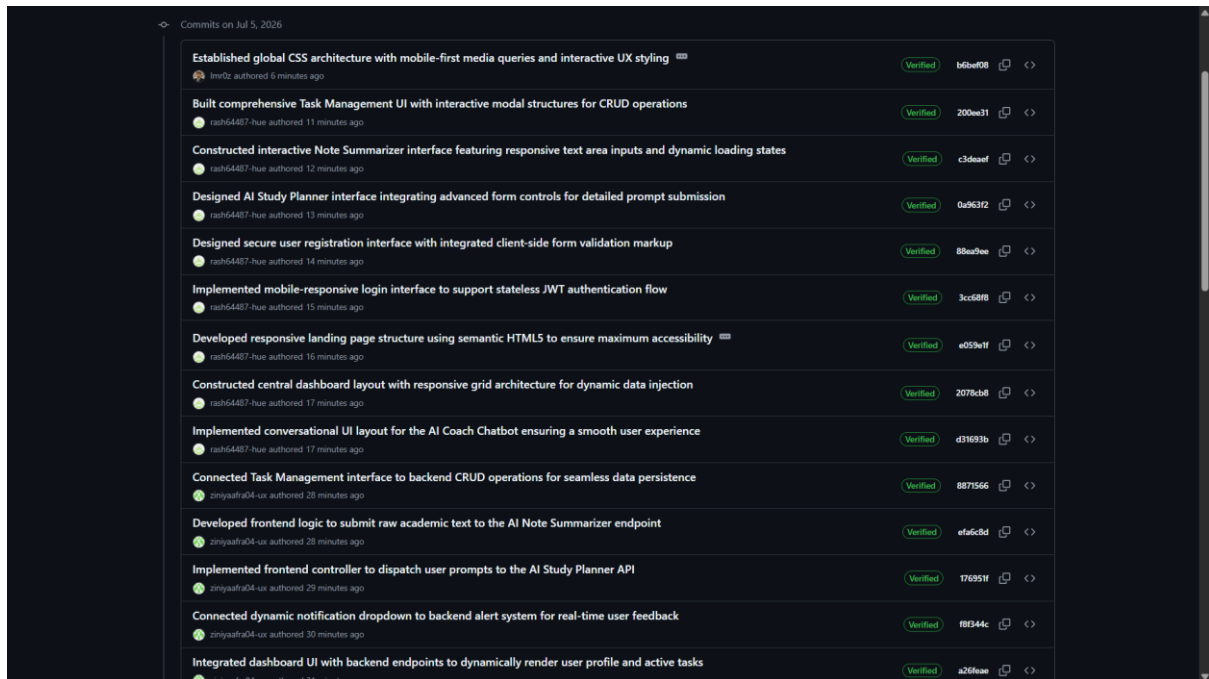
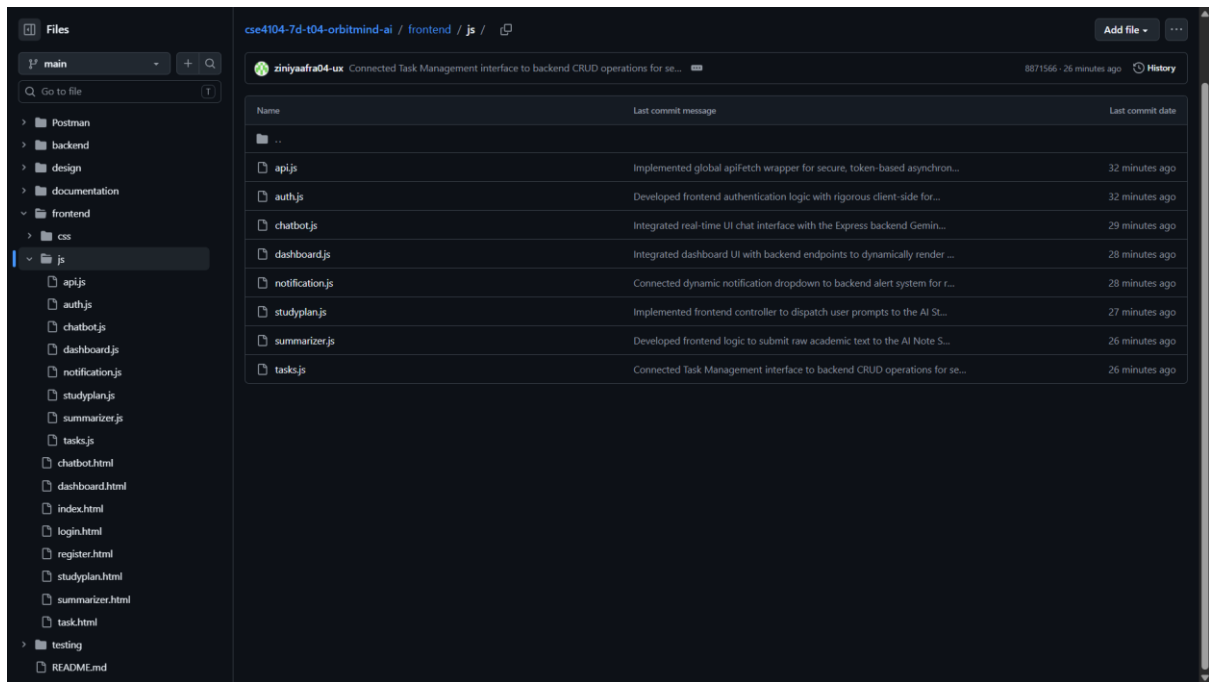


Figure 11: Maintained GitHub repository documenting continuous frontend development commits.